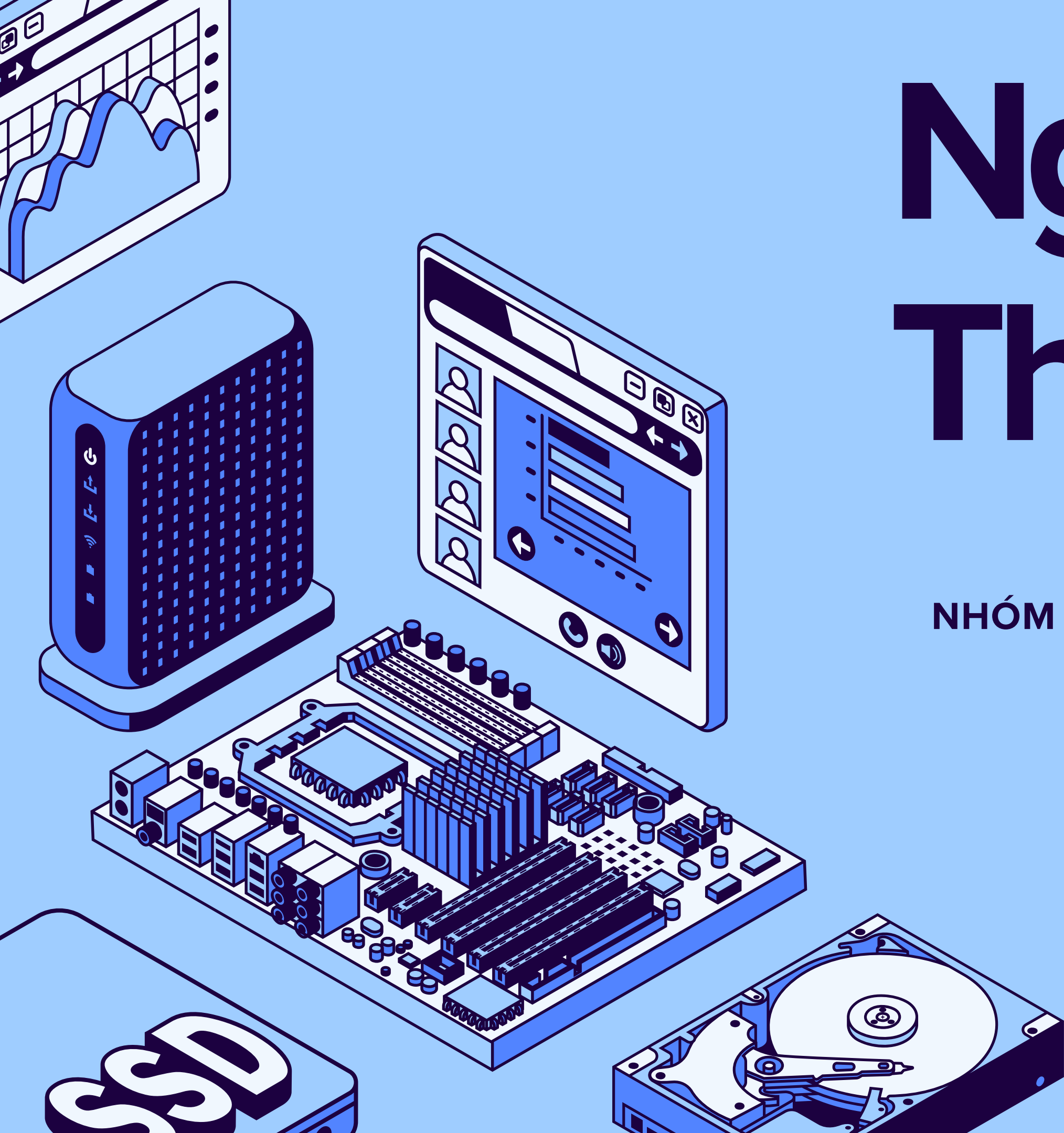


Nguyên tắc Thiết kế API

NHÓM 10



Best practices

1. Nhất quán (Consistency)
2. Rõ ràng & Dễ hiểu (Clarity)
3. Dễ mở rộng (Extensibility)



```
1  {
2    "data": [
3      {
4        "author": "sun tzu",
5        "id": 1,
6        "title": "the art of war",
7        "year": 450
8      },
9      {
10       "author": "brian kernighan",
11       "id": 2,
12       "title": "the c programming language",
13       "year": 1978
14     },
15     {
16       "author": "Unknown",
17       "id": 3,
18       "title": "dota: dragon's blood",
19       "year": 2024
20     }
21   ],
22   "metadata": {
23     "apiVersion": "1.0"
24   }
25 }
```

Response thành công luôn theo cấu trúc sau:

```
{
  "data": [...],
  "metadata": {...}
}
```

Khái niệm: Tính nhất quán đòi hỏi mọi endpoint trong hệ thống phải tuân theo một bộ quy tắc duy nhất về cách đặt tên, cấu trúc dữ liệu trả về và mã lỗi.

Ví dụ về tính thiếu nhất quán:

- Danh sách người dùng: *GET /users* (Trả về mảng JSON).
- Danh sách đồ yêu thích: *GET /wishlist-items* (Lại trả về một Object chứa mảng JSON bên trong).
- Xóa người dùng trả về HTTP 200, nhưng xóa đồ yêu thích lại trả về HTTP 204.

01. Nhất quán (Consistency)

02. Rõ ràng & Dễ hiểu (Clarity)

Khái niệm: Xây dựng API sao cho lập trình viên chỉ cần nhìn vào URL và HTTP Method là có thể dễ dàng đoán gần đúng chức năng của endpoint đó.

Ví dụ:

- *POST /system/action/do-update?type=email&id=5* (Người dùng không biết API này làm gì, cần truyền body thế nào)
- *PATCH /users/5/email* (Chỉ nhìn lướt qua là biết: Cập nhật email cho user có ID là 5).

```
@app.route('/api/v1/books', methods=['GET'])
> def get_books(): ...

@app.route('/api/v1/books/<int:book_id>', methods=['GET'])
> def get_book(book_id): ...

@app.route('/api/v1/books', methods=['POST'])
> def create_book(): ...
```

03. Dễ mở rộng (Extensibility)

```
def wrap_with_metadata(data):  
    return jsonify(  
        "metadata": {  
            "apiVersion": "1.0",  
            "timestamp": datetime.now().isoformat()  
        },  
        "data": data  
    )
```

```
1  {  
2      "data": {  
3          "author": "Unknown",  
4          "id": 3,  
5          "title": "dota: dragon's blood",  
6          "year": 2024  
7      },  
8      "metadata": {  
9          "apiVersion": "1.0",  
10         "timestamp": "2026-03-12T12:01:10.466043"  
11     }  
12 }
```

Thêm timestamp vào metadata

Khái niệm: Thiết kế API không chỉ cho hiện tại mà phải tính cho tương lai. Khi cơ sở dữ liệu phình to hoặc nghiệp vụ thay đổi, API cũ vẫn phải hoạt động bình thường cho các client chưa kịp cập nhật.

Ví dụ:

Trả về `{ "data": [...] }` thay vì `[...]`.

Về sau nếu cần thêm thông tin phân trang (pagination) vào response, chỉ cần thêm `{ "data": [...], "pagination": {...} }` mà không làm hỏng cấu trúc cũ.

Naming conventions

1. Plural Nouns

2. Lowercase & Hyphen

3. Versioning



Tài nguyên là danh từ, không phải động từ và luôn sử dụng danh từ số nhiều.

Ví dụ:

- *GET /users* (Lấy danh sách người dùng).
- *GET /users/123* (Lấy người dùng có mã 123 từ trong danh sách users).
- *GET /books* (Lấy danh sách đầu sách).
- *GET /books/1* (Lấy sách có mã 1 trong danh sách đầu sách).

```
@app.route('/api/v1/books', methods=['GET'])
> def get_books(): ...

@app.route('/api/v1/books/<int:book_id>', methods=['GET'])
> def get_book(book_id): ...

@app.route('/api/v1/books', methods=['POST'])
> def create_book(): ...
```

1. Plural Nouns

2. Lowercase & Hyphen

Kebab case: Dùng chữ cái in thường toàn bộ. Nếu tài nguyên có hai âm tiết trở lên, sử dụng dấu gạch ngang (-) để nối từ nhằm tăng tối đa khả năng đọc hiểu. Không sử dụng Snake case hay Camel case

Ví dụ:

- /users
- /punishment-stats
- /recent-games



3. Versioning

Đặt phiên bản ngay ở đầu đường dẫn.

Luôn sử dụng ký tự v kèm theo một số nguyên (Major Version).

Ví dụ:

- Hiện tại: /v1/users
 - Gồm các trường name, email, phone
- Tương lai: v2/users
 - Sau khi thay đổi cấu trúc của Data Transfer Object
 - Tách trường name thành firstName, lastName
 - Gộp trường phone và email vào contactInfo



Thank You

